5. Supplementary Material (Appendix).

5.1. Notation reference.

Table 5.1: Principal notation used in this paper. Standard graph-theoretic symbols (G, V, E, A, D, L) follow the definitions in Section 2.1.

Symbol	Description	Defined in
k	Ground-truth edge length (strong oracle)	Section 2.3, Figure 2.1
k'	Noise-perturbed edge length (weak oracle)	Section 2.5
$k_{\min}$	Minimum admissible edge length, $2r_{\max} + \varepsilon$	Section 2.3 (A2), Figure 2.1
ε	Clearance / ε-thickness separation	Section 2.3 (A1)
$r_{\max}$	Maximum admissible jump radius (clipping bound)	Section 2.2.2
r_a, r_b	Radial displacement magnitudes per vertex	Figure 2.1
c	Scale parameter of the CTPL distribution	Section 2.2.1, Section 2.2.2
λ	Exponential tempering parameter	Section 2.2.2
δ	Tail tolerance for calibration	Section 2.2.2
N_D	Doubling constant of the ambient metric	Section 2.3
φ	Radial weight kernel (edge weight as function of distance)	Section 2.6
ρ	Inter-vertex distance (scalar argument to weight function)	Section 2.3 (A4)
$w_{\max}, L_w$	Weight bound and Lipschitz constant of φ	Section 2.3 (A4)
θ	Minimum angular separation between neighbours at a hub	Section 2.3 (A2)
τ_d	Kissing number in dimension d ($\tau_2 = 6$)	Section 2.6
$\mathrm{WD}_G(v)$	Weighted degree of vertex v	Section 2.6
$\delta_{\mathrm{rel}}\mathrm{WD}(v)$	Relative weighted-degree change under perturbation	Section 2.6
C_n	Geometric sensitivity coefficient for degree-n motif	Section 2.6
G_n	Hub–spoke witness motif of degree n	Section 2.6
d_{SD}	Normalised spectral distance (Wasserstein-2 based)	Section 2.1
$S_2(L, L')$	ℓ_2 spectral shift	Section 2.8
SC_k	Stochastic co-spectrality (k-th moment of d_{SD})	Section 2.9
S3I	Stochastic spectral separation index	Section 2.9
$S(H)$	Sum of squared edge-weight changes on graph H	Section 2.8
$\Delta(H)$	Maximum vertex degree in graph H	Section 2.8

5.2. Algorithms and reproduction. Source code supporting this paper is structured as 2 Julia packages and supporting scripts:

- DynamicGeometricGraphs.jl: geometric graph constructions, repair algorithms, motif detection and tiling.
  https://github.com/systems-mechanobiology/DynamicGeometricGraphs.jl
- CalibratedTemperedPowerLaw.jl: implementation of the CTPL distribution, sampling algorithms, S3I, SC metrics, and noise perturbation routines.
  https://github.com/systems-mechanobiology/CalibratedTemperedPowerLaw.jl
- GeometricVertexNoiseSpectra: scripts to reproduce figures and experiments in the paper, including the geometric motifs and maximal configurations and angular maximizers.
  https://github.com/systems-mechanobiology/GeometricVertexNoiseSpectra

This manuscript is for review purposes only.

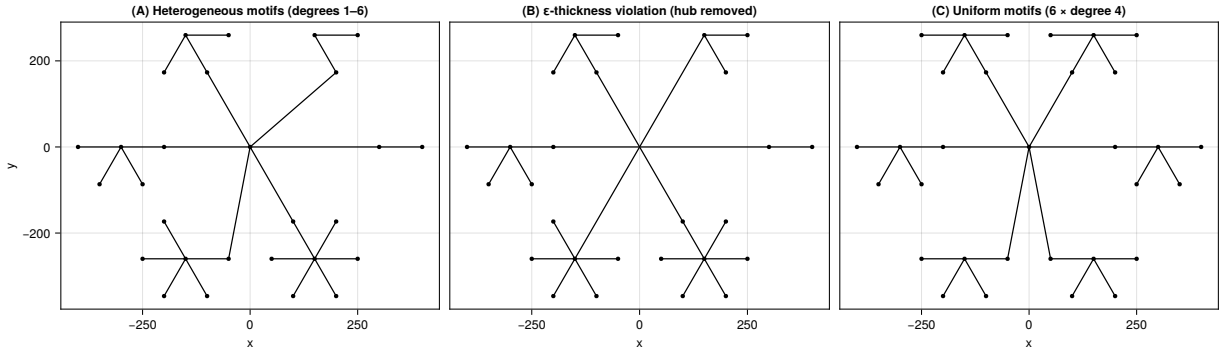

Figure 5.1: Graph constructions used for motif-frequency and repair experiments. Three geometric graph constructions sharing the same spatial scaffold (edge length $k = 100$, inter-motif spacing 300; arbitrary units). (A) Motif-complete reference graph realizing all six witness motifs under the ε-thickness constraint, used as the baseline construction in Figure 2.5. (B) Variant violating the ε-thickness constraint by removal of the central hub, illustrating the geometric collapse excluded by the model assumptions. (C) Frequency-controlled construction with uniform motif composition (six copies of the degree-4 motif), used to isolate the effect of motif frequency independently of geometry. Exact construction parameters are given in the reproduction scripts.

Algorithm 1: Sample from CTPL distribution

Input: Scale $c > 0$, tempering $\lambda \geq 0$, location μ, sample limit L
Output: Sample x from $\mathrm{CTPL}(c, \lambda, \mu)$, $x \leq L$

while *True* **do**
  Draw $z \sim \mathcal{N}(0, 1)$;
  $x_{\mathrm{cand}} \leftarrow \mu + \frac{c}{z^2}$;
  Compute proposal PDF: $p_{\mathrm{prop}} \leftarrow \mathrm{LevyPDF}(x_{\mathrm{cand}}, c, \mu)$;; `// Sec 2.2.1`
  if $p_{prop} \leq 0$ *or not finite* **then**
    continue;
  Compute target PDF: $p_{\mathrm{target}} \leftarrow \mathrm{TemperedLevyPDF}(x_{\mathrm{cand}}, c, \lambda, \mu)$;; `// Sec 2.2.2`
  $r \leftarrow \frac{p_{\mathrm{target}}}{p_{\mathrm{prop}}}$;
  Draw $u \sim \mathrm{Uniform}(0, 1)$;
  if $u < r$ **then**
    return $\min(L, x_{\mathrm{cand}})$;

Algorithm 2: Autotune λ for Tail Probability

Input: Scale c, location μ, sample limit L, target tail probability δ, tolerance ϵ, max iterations N
Output: λ such that $P(X > L) < \delta$

Set $\lambda_{\mathrm{low}} \leftarrow 0$, $\lambda_{\mathrm{high}} \leftarrow 2$ (sufficient for the parameter ranges considered here);
for $i \leftarrow 1$ *to* N **do**
  $\lambda_{\mathrm{mid}} \leftarrow (\lambda_{\mathrm{low}} + \lambda_{\mathrm{high}})/2$;
  $p_{\mathrm{tail}} \leftarrow \text{estimate-tail}(\lambda_{\mathrm{mid}}, \mu, c, M)$;
  if $|p_{tail} - \delta| < \epsilon$ **then**
    return λ_{mid};
  else if $p_{tail} > \delta$ **then**
    $\lambda_{\mathrm{low}} \leftarrow \lambda_{\mathrm{mid}}$;
  else
    $\lambda_{\mathrm{high}} \leftarrow \lambda_{\mathrm{mid}}$;
return *best* λ_{mid};

This manuscript is for review purposes only.

Algorithm 3: Estimate Tail Probability by Monte Carlo

Input: Scale c, tempering λ, location μ, threshold L, number of samples M
Output: Estimated tail probability p_{tail}

```
count ← 0;
for i ← 1 to M do
    Draw x ~ CTPL(c, λ, μ);
    if x > L then
        count ← count + 1;
p_tail ← count/M;
return p_tail;
```

Algorithm 4: Perturb 2D Vertex by CTPL Noise

Input: Coordinates $x, y \in \mathbb{R}^2$, scale c, limit $r_{\max}$, tolerance δ
Output: Perturbed coordinates x', y'

```
Tune λ using Algorithm 2 so that P[r ~ CTPL(c, λ) ≥ r_max] < δ;
Draw r ~ CTPL(c, λ);
Draw θ ~ Uniform[0, 2π);
x' ← x + r cos θ;
y' ← y + r sin θ;
return (x', y');
```

Algorithm 5: Motif-Extractor: Greedy selection of disjoint fragile motifs. Motifs are sorted by Table 2.1: process largest C_d

Input: $G = (V, E)$ (graph), D (degree vector), B (binary adjacency matrix), degree_priority $= \{6, 5, 4, 3, 2, 1\}$
Output: $\mathcal{M}$ (set of disjoint motif hub vertices)

```
M ← ∅;
Covered ← ∅;
foreach d in degree_priority do
    candidates ← {v ∈ V | D[v] = d};
    foreach v in candidates do
        if v ∉ Covered then
            M_cand ← {v} ∪ {u ∈ V | B[v, u] = 1};
            if M_cand ∩ Covered = ∅ then
                M ← M ∪ {v};
                Covered ← Covered ∪ M_cand;
return M;
```

Algorithm 6: ε-Thickness Repair

Input: Graph $G = (V, E)$ embedded in $\mathbb{R}^d$, separation threshold $\varepsilon > 0$
Output: Repaired graph $\widetilde{G} = (V \cup V_s, \widetilde{E})$ satisfying ε-thickness

```
Q ← E;
while Q ≠ ∅ do
    e = (u, w) ← Q.Dequeue();
    V_close ← {v ∈ V \ {u, w} : d(v, segment(e)) < ε};
    if V_close ≠ ∅ then
        Choose v ∈ V_close minimizing d(v, segment(e));
        E ← E \ {(u, w)};
        E ← E ∪ {(u, v), (v, w)};
        Q.Enqueue((u, v));
        Q.Enqueue((v, w));
    else
        // Edge e satisfies thickness constraint, keep it
return G̃ = (V ∪ V_s, Ẽ)
```

5.3. Proof that optimal motif tiling is NP-hard. We formalise the optimal motif tiling problem and relate it to the maximum weight independent set (MWIS) problem on a derived conflict graph.

Let $G = (V, E)$ be a fixed graph and let $\mathcal{M} = \{M_1, \ldots, M_m\}$ be a finite family of candidate motifs, where each motif is a vertex subset $M_i \subseteq V$. A feasible motif tiling is a subcollection $\mathcal{T} \subseteq \mathcal{M}$ such that the motifs are vertex-disjoint, i.e., $M_i \cap M_j = \varnothing$ for all $M_i, M_j \in \mathcal{T}$, $i \neq j$. We define the coverage of a tiling as the total number of vertices covered, $\mathrm{cov}(\mathcal{T}) = |\bigcup_{M_i \in \mathcal{T}} M_i| = \sum_{M_i \in \mathcal{T}} |M_i|$, where the last equality uses disjointness. The optimal motif tiling problem is to maximise $\mathrm{cov}(\mathcal{T})$ subject to $\mathcal{T} \subseteq \mathcal{M}$ and $M_i \cap M_j = \varnothing$.

From $\mathcal{M}$ we build a conflict graph $H = (V_H, E_H)$ as follows: for each motif $M_i \in \mathcal{M}$, introduce a vertex $v_i \in V_H$; connect v_i and v_j by an edge in H whenever the corresponding motifs overlap, i.e., $\{v_i, v_j\} \in E_H \iff M_i \cap M_j \neq \varnothing$; and assign each vertex v_i a non-negative weight $w_i = |M_i|$.

An independent set $I \subseteq V_H$ is a set of vertices with no internal edges. By construction, I is independent

This manuscript is for review purposes only.

if and only if $\mathcal{T}_I = \{M_i : v_i \in I\}$ is a feasible tiling. The weight of an independent set I is $w(I) = \sum_{v_i \in I} w_i = \sum_{M_i \in \mathcal{T}_I} |M_i| = \mathrm{cov}(\mathcal{T}_I)$. Thus the optimal motif tiling problem is equivalent to the maximum weight independent set problem on H: $\max_{\mathcal{T} \text{ feasible}} \mathrm{cov}(\mathcal{T}) \iff \max_{I \text{ independent in } H} w(I)$.

The maximum weight independent set problem is NP-hard on general graphs. Because any instance of MWIS can be represented as the conflict graph H of a suitable motif family (via the construction above), solving the optimal motif tiling problem is at least as hard as solving MWIS. Consequently, the optimal motif tiling problem is NP-hard in general.

AI usage disclaimer. During the preparation of this manuscript, we used generative AI tools to assist with implementation tasks (e.g., boilerplate code generation and plotting scripts), routine code screening, and publication logistics (Quarto setup and formatting). All substantive intellectual contributions, including problem formulation, theory development, proofs, interpretation of results, and the final textual content, are the result of human scholarly effort. The authors take full responsibility for the accuracy and integrity of the final manuscript.

This manuscript is for review purposes only.